Registro de interacciones con ChatGPT

TFM 14MBID | T-049 | Detección automática de anomalías en curvas de ensayos de materiales mediante técnicas de Machine Learning

EVIDENCIA TEMÁTICA — CONSULTAS, REVISIÓN METODOLÓGICA, AUDITORÍA TÉCNICA Y REPRODUCIBILIDAD

Alcance, fuentes y criterios editoriales

Este documento presenta una selección temática ampliada de interacciones conservadas en dos materiales aportados por el estudiante. Se realizaron consultas técnicas y revisión de procedimientos. No se fabrican preguntas ni se alteran las intervenciones para hacerlas parecer originales.

La transcripción conserva la redacción original en la medida en que se encuentra disponible. Se retiran únicamente marcas de interfaz repetidas (p. ej., «User attachment»). Las formulaciones no verificables del estudiante no se recrean.

Índice de contenido

- 1. Consultas de fuentes y caracterización de datasets
- 2. Auditorías técnicas del notebook y revisión de curvas
- 3. Particiones agrupadas y prevención de fuga de información
- 4. Comprobación de resultados y separación de prueba
- 5. GitHub, Google Colab y presentación del código
- 6. Revisión de anexos y notación de la memoria

1. Consultas de fuentes y caracterización de datasets

1.1. Consulta sobre repositorios de datos experimentales

Pregunta literal del estudiante: Lito me gustaria que me aydues a buscar en la web, potenciales paginas onde pueda econtrar los datasets que se podrian usar apra el análisis de curvas en resistencia de materiales y que tenga relación al tema principal de poder identificar curvas anomalas.

Objeto de la consulta: Consulta de selección y comparación de fuentes de datos; texto reproducido desde la exportación de la conversación.

Respuesta de ChatGPT (texto conservado; selección literal): Sí. Hice una búsqueda específica pensando en el T-049, no simplemente en "datasets de materiales". Busqué conjuntos que contengan curvas de ensayos mecánicos, stress–strain, load–displacement, tensile/compression, defectos y, especialmente, anomalías etiquetadas.

Encontré algo especialmente importante: existe un dataset publicado recientemente que encaja casi directamente con la finalidad de nuestro TFM.

Datasets y repositorios que priorizaría

Prioridad Dataset / portal Qué contiene Utilidad para T-049

1 — Excelente Materials Data Facility — Ground truth anomaly dataset 9.600 curvas load–displacement de nanoindentación, correspondientes a 150 muestras NiFe, anotadas por un experto y con taxonomía de anomalías Muy alta

Prioridad Dataset / portal Qué contiene Utilidad para T-049

2 — Excelente Mendeley / Scientific Data — ASTM D638 con defectos internos Curvas de tracción de PLA: tiempo, extensión, carga, strain y stress; especímenes normales y especímenes con defectos internos conocidos Muy alta

3 — Muy buena NIST — Numisheet 2020 Gran cantidad de archivos CSV de curvas tensión-deformación de ensayos de tracción y tracción/compresión sobre aceros y aluminios Alta

4 — Muy buena Zenodo — Ti-6Al-4V AM 168 curvas stress–strain más parámetros de proceso, microestructura y defectos de porosidad Alta

5 — Muy buena Mendeley — 690 ensayos de tracción, con Alta
Carbon/glass fibres diagramas load–displacement
y características del espécimen

6— Mendeley — Al Curvas engineering stress– Media-alta
Complementaria 6061-T651 strain de varios lotes,
temperaturas y estados de
tensión

7— Kaggle Stress316L Curvas stress–strain de acero Media
Complementaria inoxidable 316L a diferentes
strain rates

Ahora te explico por qué los primeros son especialmente interesantes.

1. El candidato más fuerte que he encontrado

A Ground Truth Dataset for Anomaly Detection on Load–Displacement
Nanoindentation Curves
Este dataset es excepcionalmente relevante porque fue creado específicamente para desarrollar y validar algoritmos de detección automática de anomalías en curvas de ensayos de materiales.

Contiene 9.600 curvas load–displacement, procedentes de 150 muestras de películas delgadas de NiFe, y las curvas fueron revisadas y anotadas por un especialista. El trabajo asociado las organiza en siete tipos principales de anomalía, no simplemente normal/anómala. Springer +1

Entre las categorías aparecen:

force offset;
displacement offset;
unusual unloading curvature;
unusual loading curvature;
tip displacement at holding;
too deep;
fully anomalous sample. Springer

El punto clave es que los autores desarrollaron el dataset precisamente porque la inspección manual de grandes cantidades de curvas se vuelve impracticable y querían disponer de un ground truth para Machine Learning. Además, incluye comentarios originales del experto y etiquetas categóricas. Springer

Esto se parece muchísimo a nuestra definición del T-049:

curva experimental → identificación automática de comportamiento anómalo → clasificación potencial del tipo de anomalía.

La diferencia es que aquí se trata de nanoindentación, no de tracción/compresión/flexión convencional. Pero sigue siendo un ensayo mecánico de materiales basado en curvas load–displacement, por lo que metodológicamente el ajuste es muy fuerte.

El artículo científico que documenta el dataset fue publicado en Journal of Materials Research en julio de 2026 y el dataset está disponible públicamente mediante Materials

Data Facility. Springer

Acceder al dataset de anomalías en Materials Data Facility

Consultar el artículo científico que documenta el dataset

Incluso publicaron el código Python utilizado para revisar y analizar las anomalías, lo cual puede servirnos como referencia metodológica, aunque obviamente no deberíamos limitar el TFM a reproducir su solución. Springer

Este es, a día de hoy, mi candidato nº 1 para estudiar seriamente.

2. Dataset de ensayos de tracción con especímenes normales y defectuosos

El segundo candidato es también muy interesante porque se acerca más a los ensayos indicados originalmente en T-049.

Es un dataset de especímenes ASTM D638 fabricados en PLA mediante fabricación aditiva. Existen especímenes sin defectos y otros a los que se introdujeron intencionadamente diferentes defectos internos.

Los archivos contienen por ensayo:

time, extension, load, tensile strain y stress, además de dimensiones, UTS y strain at UTS. Por tanto, tenemos las curvas completas stress–strain, no solamente propiedades finales. PubMed Central (PMC)
+1

Los defectos fueron introducidos variando geometrías, tamaños y posiciones internas, por lo que tenemos una condición experimental conocida contra la que comparar los modelos. Nature

Esto permitiría estudiar, por ejemplo:

sin defecto → comportamiento de referencia

frente a

con defecto → desviación de la curva de tracción.

Eso abre una posibilidad muy atractiva para aplicar tanto aprendizaje supervisado como detección no supervisada.

Artículo y descripción completa del dataset ASTM D638

Desde el punto de vista de los ensayos mencionados por VIU, este dataset es incluso más intuitivo de explicar que el de nanoindentación.

3. NIST — posiblemente nuestra mejor fuente de curvas mecánicas “limpias”

El National Institute of Standards and Technology (NIST) dispone del conjunto de datos de Numisheet 2020, que incluye ensayos de tracción y tensión/compresión.

Incluye cuatro materiales:

DP980;
DP1180;
aleación de aluminio 6xxx-T4;
aleación de aluminio 6xxx-T81.

Y lo más importante para nosotros es que existen numerosos archivos individuales en CSV con Stress–Strain, además de metadatos. El catálogo registra 233 recursos asociados al conjunto. Data.gov

Dataset NIST Numisheet 2020 de tracción y tensión/compresión

Aquí hay un enfoque interesante para el TFM: utilizar estas curvas como población de referencia y aplicar detección no supervisada de anomalías, o introducir anomalías controladas para estudiar la sensibilidad de diferentes algoritmos.

Pero habría que justificar científicamente cualquier generación sintética de anomalías; no podemos simplemente alterar datos arbitrariamente.

4. Ti-6Al-4V: 168 curvas + defectos del material
Este dataset de Zenodo me parece particularmente interesante para una segunda línea experimental.

Contiene datos de fabricación aditiva de Ti-6Al-4V y ofrece:

curvas stress–strain;
propiedades mecánicas;
parámetros de procesamiento;
microestructura;
rugosidad;
morfología de poros/defectos obtenida por tomografía X-ray.

El repositorio indica que existen 42 condiciones de procesamiento × 4 especímenes = 168 curvas engineering stress–strain. Zenodo

Aquí podríamos investigar algo muy interesante:

defectos reales del material ↔ modificación del comportamiento de la curva.

Eso conecta directamente con una de las finalidades originales del T-049: detectar “comportamiento atípico del material”.

Dataset Ti-6Al-4V con curvas stress–strain y defectos

5. Dataset grande de 690 ensayos de tracción
En Mendeley Data encontré un conjunto especialmente útil para ML porque contiene 690 fibras ensayadas, correspondientes a cuatro tipos de fibra de carbono y una de vidrio.

Los datos originales incluyen:

diámetro + gauge length + diagramas load–displacement.

Posteriormente pueden convertirse a stress–strain aplicando la corrección correspondiente. ScienceDirect +1

La cantidad de especímenes es mucho más interesante para Machine Learning que datasets experimentales que solo contienen 5 o 10 ensayos.

Dataset de 690 ensayos de fibras de carbono y vidrio

6. Otro buen dataset: aluminio 6061-T651
Mendeley tiene un conjunto de curvas de aluminio 6061-T651 procedentes de 9 lotes comerciales, ensayados a 6 temperaturas diferentes, tanto en tensión uniaxial como plane-strain tension.

Incluye curvas completas engineering stress–engineering strain. Mendeley Data

Este tipo de dataset puede ser interesante porque la variabilidad debida a:

lote + temperatura + espécimen

nos permite estudiar cuándo una curva deja de comportarse como las demás bajo condiciones equivalentes.

Dataset Al 6061-T651 stress–strain

Las páginas que deberíamos vigilar durante el TFM

Más allá de esos datasets concretos, hay cuatro repositorios que yo establecería como fuentes principales:

Materials Data Facility (MDF) es probablemente el más interesante para nuestro caso porque está especializado en ciencia e ingeniería de materiales y ya encontramos allí el dataset de anomalías de 9.600 curvas. Su buscador contiene categorías como machine learning, metals and alloys, nanoindentation, experiment, etc. ACDC +1

Mendeley Data está dando resultados excelentes para tensile testing , stress-strain , load-displacement , mechanical testing y materiales específicos. Allí encontramos tanto las 690 fibras como el aluminio y otros datasets de ensayos experimentales. Mendeley Data +1

Zenodo también es especialmente útil para datos experimentales acompañados de artículos científicos y DOI. Encontramos allí desde datasets Ti-6Al-4V hasta datos brutos de fractura y load–displacement. Zenodo +1

NIST Data tiene la ventaja de ser una fuente institucional muy sólida y ofrecer directamente datos experimentales bien documentados y numerosos CSV. NIST +1

Kaggle también puede utilizarse como fuente exploratoria. Por ejemplo, encontré Stress316L , basado en ensayos uniaxiales de acero inoxidable 316L a diferentes strain rates, pero por tamaño y trazabilidad científica lo pondría por detrás de MDF, NIST, Zenodo y Mendeley para un TFM. Kaggle

Mi recomendación para T-049 cambia bastante después de esta búsqueda
Veo una posibilidad metodológica mucho más fuerte de la que teníamos inicialmente.

Podríamos utilizar dos conjuntos complementarios:

Dataset principal: las 9.600 curvas expert-labelled de nanoindentación para construir y comparar rigurosamente modelos de detección/clasificación de anomalías.

Dataset de validación o transferencia: uno de los datasets de tracción —ASTM D638, NIST, Ti-6Al-4V o fibras— para estudiar si el enfoque puede trasladarse a otro tipo de ensayo mecánico.

Esto sería bastante más potente académicamente que utilizar un único CSV pequeño.

Y hay otra ventaja: el primer dataset contiene ground truth humano, por lo que podríamos comparar de verdad algoritmos supervisados, semi-supervisados y no

supervisados mediante métricas como precision, recall, F1, ROC-AUC, PR-AUC o matrices de confusión, según cómo terminemos formulando el problema. El propio trabajo de 2026 señala que disponer de estas etiquetas permite precisamente desarrollar y validar algoritmos supervisados y semi-supervisados de detección de anomalías. Springer

No tomaría todavía una decisión definitiva. El siguiente paso que recomiendo es que hagamos una evaluación técnica formal de los 5 primeros candidatos, revisando para cada dataset: tamaño real, estructura de archivos, variables, etiquetas, balance normal/anómalo, licencia, formato, facilidad de descarga, calidad documental y qué modelos permitiría probar. De ahí podremos seleccionar objetivamente cuál sería el dataset principal del T-049.

Thu, Sep 17 at 11:23 AM

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

1.2. Consulta sobre datasets de materiales en Colombia

Pregunta literal del estudiante: Busca en la Web datasets de ensayos a materiales en Colombia.

Objeto de la consulta: Consulta de selección y comparación de fuentes de datos; texto reproducido desde la exportación de la conversación.

Respuesta de ChatGPT (texto conservado; selección literal): Sí. Hice una búsqueda centrada específicamente en datasets públicos asociados a
instituciones colombianas o ensayos realizados en Colombia, y encontré varios que pueden ser útiles para el TFM T-049. Los más relevantes no están en Kaggle, sino principalmente en Mendeley Data y repositorios institucionales.

Dataset	Institución colombiano	Ensayo / datos	Ajuste al T-049
Monotonic dataset for 3D printed composites	Universidad Industrial de Santander	Tracción monotónica hasta rotura; archivos .txt con tiempo, desplazamiento y fuerza	Muy alto
Tensile test data on	Universidad	Ensayos de tracción	Muy alto

PLA samples with and without sterilization	Industrial de Santander	de PLA; datos brutos en Excel	
Dataset colombiano	Institución	Ensayo / datos	Ajuste al T-049
Physical and Mechanical Properties of a Dental Resin Composite for 3D Printing	Universidad de Antioquia	Flexión, compresión, dureza y otras propiedades; archivos Excel	Alto
Myricetin as a Radioprotective Biomodifier...	Universidad Nacional de Colombia / Fundación Santa Fe	Flexión a tres puntos, microtracción y curvas stress–strain de dentina	Alto
Influence of seismic porewater pressure...	Universidad Cooperativa de Colombia	Base de ensayos triaxiales cíclicos y corte simple; comportamiento stress–strain	Medio-alto
Resultados de Análisis de Laboratorio Suelos en Colombia	AGROSAVIA	Resultados de laboratorio de suelos a nivel nacional	Bajo para curvas mecánicas, alto para detección de anomalías

1. Universidad Industrial de Santander: composites impresos en 3D
Este es probablemente el dataset colombiano más interesante que encontré para T-049.

El conjunto contiene ensayos de tracción monotónica bajo desplazamiento controlado hasta la rotura de materiales compuestos termoplásticos impresos en 3D y reforzados con fibras continuas. Los archivos están publicados como texto plano y contienen directamente:

tiempo → desplazamiento → fuerza

Esto nos permitiría construir las curvas experimentales y aplicar detección de anomalías directamente sobre su forma. El dataset está asociado a la Universidad Industrial de Santander, es descargable y tiene licencia CC BY 4.0. Mendeley Data

Dataset UIS: Monotonic dataset for 3D printed composites

Para nuestro TFM esto tiene una ventaja enorme: no estamos trabajando únicamente con parámetros finales como resistencia máxima o módulo de Young, sino con las señales originales de los ensayos, que es precisamente lo que necesitamos para analizar curvas.

2. Universidad Industrial de Santander: PLA con y sin esterilización
Encontré otro conjunto colombiano de la UIS bastante sólido. Contiene resultados de

ensayos de tracción de piezas estándar de PLA fabricadas mediante FDM, comparando muestras de control frente a muestras sometidas a esterilización.

Lo importante es que Mendeley confirma que incluye datos brutos de tracción en archivos Excel, además de código en R para análisis estadístico. Los especímenes fueron evaluados en un contexto experimental vinculado a Bucaramanga y la investigación fue financiada por Minciencias. Tiene licencia CC BY 4.0. Mendeley Data

Dataset UIS: Tensile test data on PLA samples with and without sterilization

Este también podría servirnos muy bien para investigar si las curvas correspondientes a diferentes condiciones presentan patrones suficientemente diferenciables mediante Machine Learning.

3. Universidad de Antioquia: flexión y compresión de resinas
Un dataset colombiano muy reciente, publicado en marzo de 2026, procede de la Universidad de Antioquia, Medellín.

Contiene ensayos sobre una resina compuesta para impresión 3D con diferentes porcentajes de refuerzo cerámico. Incluye datos de:

resistencia y módulo a flexión;
resistencia a compresión;
dureza;
absorción de agua;
propiedades físicas adicionales.

Los archivos se distribuyen en Excel, incluyendo archivos específicos de resultados mecánicos y gráficos. Tiene licencia CC BY 4.0. Mendeley Data

Dataset Universidad de Antioquia: Physical and Mechanical Properties of a Dental Resin Composite

Para T-049 lo considero interesante, aunque por ahora lo situaría por debajo de los dos datasets de UIS, porque no puedo confirmar todavía que todos los registros contengan las curvas completas punto a punto y no únicamente propiedades derivadas.

4. Universidad Nacional de Colombia: curvas stress–strain y flexión
Encontré también un dataset de 2026 de la Universidad Nacional de Colombia en Bogotá, en colaboración con la Fundación Santa Fe.

Aunque el material es dentina humana, desde el punto de vista mecánico es bastante interesante. Se realizaron ensayos de flexión a tres puntos, calculando resistencia, deformación y módulo elástico a partir de curvas stress–strain, además de ensayos de microtracción. Los datos brutos se encuentran en hojas de cálculo organizadas por grupo experimental. Mendeley Data

Dataset Universidad Nacional: Myricetin as a Radioprotective Biomodifier dataset

Además, existen condiciones experimentales diferenciadas —control, distintas dosis de radiación y tratamiento—, lo que puede ser útil para estudiar detección de cambios o comportamientos atípicos.

5. Universidad Cooperativa de Colombia: 116 ensayos cíclicos

Encontré un dataset asociado a la Universidad Cooperativa de Colombia centrado en comportamiento geotécnico.

La investigación utiliza una base de 116 ensayos triaxiales cíclicos y ensayos de corte simple cíclico, estudiando curvas stress–strain, presión de poros, ablandamiento y licuefacción. El dataset está publicado en Mendeley con licencia CC BY 4.0. Mendeley Data

Dataset Universidad Cooperativa de Colombia

No es exactamente el tipo de material que probablemente imaginaba el tema T-049 — metales, polímeros o composites—, pero metodológicamente es relevante porque trabaja con curvas mecánicas experimentales y comportamiento anómalo/no lineal.

6. AGROSAVIA: un caso colombiano interesante para detección de anomalías

Encontré además un dataset nacional oficial llamado Resultados de Análisis de Laboratorio Suelos en Colombia, publicado por AGROSAVIA. Contiene resultados del laboratorio de química y física de suelos de la entidad para muestras procedentes de distintas regiones del país. Agrosavia

Dataset AGROSAVIA: Resultados de Análisis de Laboratorio Suelos en Colombia

No contiene curvas de tracción/compresión, por lo que no lo elegiría para T-049. Sin embargo, encontré algo metodológicamente muy interesante: en 2025 se desarrollaron

proyectos colombianos sobre ese dataset que emplearon Isolation Forest, Z-score robusto, IQR y K-Means para detectar anomalías y valores atípicos. Datos Abiertos +1

Eso podría servirnos más adelante como referencia colombiana de aplicación de anomaly detection sobre datos experimentales de laboratorio, aunque no como dataset principal.

De los colombianos, hay dos que destacan claramente

Para T-049 — Detección automática de anomalías en curvas de ensayos de materiales mediante Machine Learning, yo pondría ahora mismo en primera línea estos dos:

A. UIS — Monotonic dataset for 3D printed composites

Porque tenemos directamente tiempo + desplazamiento + fuerza , múltiples configuraciones de material y ensayo hasta rotura. Eso es prácticamente la materia prima que necesitamos para generar y analizar curvas.

B. UIS — PLA tensile tests with/without sterilization

Porque también contiene datos experimentales brutos de tracción y condiciones de ensayo diferenciadas.

Incluso hay una posibilidad que me parece científicamente atractiva: utilizar un dataset colombiano como dataset principal y un dataset internacional etiquetado específicamente para anomalías como benchmark externo. Así el TFM tendría una conexión colombiana real sin renunciar a disponer de un conjunto con ground truth para validar los algoritmos.

Mi siguiente paso sería descargar y abrir los dos datasets de la UIS, contar exactamente cuántas curvas contienen, revisar el número de puntos por curva, comprobar

si hay curvas aparentemente anómalas y determinar si las condiciones experimentales pueden utilizarse como etiquetas. Ahí sí podríamos saber cuál de los dos tiene suficiente volumen y calidad para convertirse en el dataset principal del TFM.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

2. Auditorías técnicas del notebook y revisión de curvas

2.1. Validación de la etapa de mantenimiento de nanoindentación

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Revisión de fases de carga, mantenimiento y descarga; verificación de la temporización; exportación de figura y observaciones sobre curvas normales/anómalas.

Respuesta de ChatGPT (texto conservado; selección literal): Perfecto. Aquí acabamos de obtener una de las evidencias experimentales más claras del TFM.

El protocolo específico de este dataset utiliza un perfil acelerado de 0,1 s de carga + 0,2 s de mantenimiento + 0,1 s de descarga, por lo que una duración total cercana a 0,4 s, como los 0,398362 s que obtuviste, es exactamente coherente con el experimento de los autores.

Además, nuestro criterio automático de carga ≥99 % del máximo identificó 203 puntos aproximadamente entre 0,10 y 0,30 s, es decir, prácticamente el intervalo de mantenimiento del protocolo. Esto valida muy bien la extracción.

Y el resultado cuantitativo es todavía mejor:

$$\Delta h_{normal} = +5.9180\ nm$$

$$\Delta h_{anómala} = -4.1939\ nm$$

La curva normal continúa penetrando durante el mantenimiento, mientras que la anómala alcanza un máximo y luego retrocede en profundidad. Los autores indican precisamente que, para estas películas de NiFe, bajo carga constante la profundidad debería aumentar por efectos dependientes del tiempo; la anomalía tip_displacement_decreases_at_holding aparece cuando ocurre el comportamiento contrario.

Por tanto:

TFM-03C — VALIDADA ✅
Evolución de la profundidad durante la etapa de mantenimiento.
Destino recomendado: Capítulo 4 — análisis exploratorio, como figura acompañada de la comparación cuantitativa.

Y esta figura es incluso más demostrativa que el panel general para esta anomalía concreta.

Guarda también la figura
Añade:

```
# CELDA 27 — Exportar TFM-03C

fig_path = (
OUTPUT_DIR
/ "TFM_03C_mantenimiento_normal_anomalo.png"
)

fig.savefig(
fig_path,
dpi=300,
bbox_inches="tight"
)

print("Figura guardada en:")
print(fig_path)
```

Para la memoria podremos redactar posteriormente algo parecido a:

Durante la etapa de mantenimiento se observó una diferencia clara entre las curvas analizadas. En el ejemplo normal, la profundidad aumentó 5,918 nm entre el comienzo y el final del intervalo considerado, mientras que en la curva etiquetada como anómala disminuyó 4,194 nm. Este comportamiento es coherente con la categoría de anomalía definida por Zakariya et al. (2026), asociada a una disminución del desplazamiento de la punta durante el mantenimiento de carga.

Aquí cada cifra procede directamente de tu notebook, así que queda perfectamente trazable.

TFM-04 — Distribución de anomalías por muestra
Ahora tenemos que abordar un aspecto crítico antes de dividir los datos en entrenamiento y prueba.

Sabemos que cada muestra contiene 64 indentaciones. Necesitamos comprobar experimentalmente cómo se distribuyen las 3.409 curvas anómalas entre las 150 muestras.

Esto determinará cómo vamos a evitar data leakage.

CELDA 28 — Resumen por muestra

```
# ============================================================
# TFM-04
# Distribución de anomalías por muestra
# ============================================================

sample_summary = (
labels
.groupby("sample_num")
```

```
.agg(
total_curves=("indent_num", "size"),
anomalous_curves=("is_anomaly", "sum")
)
.reset_index()
)

sample_summary["normal_curves"] = (
sample_summary["total_curves"]
- sample_summary["anomalous_curves"]
)

sample_summary["anomaly_percentage"] = (
sample_summary["anomalous_curves"]
/ sample_summary["total_curves"]
* 100
)

display(sample_summary.head())

CELDA 29 — Casos extremos
samples_without_anomalies = (
sample_summary["anomalous_curves"] == 0
).sum()

fully_anomalous_samples = (
sample_summary["anomalous_curves"]
== sample_summary["total_curves"]
).sum()

print(
"Muestras sin curvas anómalas:",
samples_without_anomalies
)

print(
"Muestras con las 64 curvas anómalas:",
fully_anomalous_samples
)

print("\nMuestras completamente anómalas:")

display(
sample_summary[
sample_summary["anomalous_curves"]
== sample_summary["total_curves"]
]
)
```

De nuestra auditoría preliminar espero:

0 muestras completamente normales
11 muestras con las 64 curvas anómalas

pero quiero que Colab lo demuestre.

CELDA 30 — Distribución gráfica por muestra

```
fig, ax = plt.subplots(figsize=(9, 5))

ax.hist(
sample_summary["anomaly_percentage"],
bins=15
)

ax.set_xlabel(
"Porcentaje de curvas anómalas por muestra (%)"
)

ax.set_ylabel(
"Número de muestras"
)

ax.set_title(
"Distribución de la proporción de anomalías por muestra"
)

ax.grid(
axis="y",
alpha=0.25
)

plt.tight_layout()
plt.show()
```

Esta será provisionalmente:

FIGURA TFM-04A — Distribución de la proporción de curvas anómalas por muestra.

CELDA 31 — Comprobación clave para la futura partición

```
print(
"Muestras con al menos una curva normal:",
(sample_summary["normal_curves"] > 0).sum()
)

print(
"Muestras con al menos una curva anómala:",
(sample_summary["anomalous_curves"] > 0).sum()
)

print(
"Muestras con ambas clases:",
(
(sample_summary["normal_curves"] > 0)
&
(sample_summary["anomalous_curves"] > 0)
).sum()
)
```

Esto es importante porque después podremos decidir correctamente entre:

una única partición train / validation / test por muestras;
o, preferiblemente, validación cruzada agrupada por sample_num.
Todavía no hagamos la división.

Primero quiero ver cómo están distribuidas las anomalías entre las muestras. Si hacemos una partición por grupos sin mirar esto, podríamos crear un conjunto de prueba con una distribución artificialmente distinta a la de entrenamiento.

Lo que ya hemos conseguido
Hasta este momento:

TFM-01 ✅ Integridad y estructura.
TFM-02 ✅ Distribución de anomalías.
TFM-03A/B/C ✅ Inspección visual y comportamiento físico.
TFM-04 🔄 Distribución por muestra y preparación para partición.

Y hay una conclusión metodológica que ya podremos defender con evidencia: no debemos tratar las 9.600 curvas como 9.600 observaciones completamente independientes al dividir train/test, porque están organizadas en grupos de 64 indentaciones correspondientes a 150 muestras físicas. Los propios autores formalizan el dataset precisamente como 150 conjuntos de muestras, cada uno compuesto por 64 indentaciones.

Ejecuta ahora las celdas 28–31 y mándame los resultados de 28, 29, 30 y 31. Con eso decidiremos la estrategia exacta de partición antes de construir nuestros primeros modelos.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

2.2. Revisión de estructura del dataset y estrategia de análisis

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Comprobaciones de integridad y revisión del procedimiento experimental.

Respuesta de ChatGPT (texto conservado; selección literal): Perfecto. TFM-04 queda validada ✅ y nos da justo la información necesaria para tomar una decisión correcta de partición.

Tus resultados son bastante claros:

150 muestras.
Cada una contiene 64 curvas.
Mínimo de anomalías por muestra: 8/64 = 12,5 %.
Máximo: 64/64 = 100 %.
Mediana: 20/64 = 31,25 %.
Media: 22,73/64 ≈ 35,52 %, prácticamente idéntica al 35,51 % global.
0 muestras completamente normales.
11 muestras completamente anómalas.
139 muestras contienen simultáneamente curvas normales y anómalas.
Las 150 muestras tienen al menos una curva anómala.
La figura también muestra algo importante: la mayoría de las muestras está entre aproximadamente 15–45 % de anomalías, mientras existe un grupo claramente separado de 11 muestras al 100 %.

Esto significa que una división aleatoria a nivel de curva sería metodológicamente incorrecta: podríamos entrenar con

algunas indentaciones de una muestra y evaluar con otras indentaciones de esa misma muestra física.

Decisión de partición
Vamos a utilizar StratifiedGroupKFold, que intenta conservar la proporción de clases pero obliga a que cada grupo permanezca íntegramente en un único subconjunto. En nuestro caso:

y = normal/anómala;
groups = sample_num.
Es exactamente el problema para el que está diseñado este esquema. La documentación actual de scikit-learn especifica que StratifiedGroupKFold intenta preservar la proporción de clases manteniendo grupos no solapados entre folds.

Pero antes de fijar definitivamente train/test, vamos a auditar los cinco folds. No quiero asumir que quedan equilibrados, especialmente por esas 11 muestras completamente anómalas.

Guardar TFM-04A
Si todavía no lo hiciste:

CELDA 32 — Exportar figura TFM-04A
```
fig_path = (
OUTPUT_DIR
/ "TFM_04A_distribucion_anomalias_por_muestra.png"
)

fig.savefig(
fig_path,
dpi=300,
bbox_inches="tight"
)

print("Figura guardada en:")
print(fig_path)
```

Registro:

FIGURA TFM-04A — VALIDADA ✅
Distribución de la proporción de curvas anómalas por muestra.
Destino: Capítulo 4 → estrategia de partición / caracterización del dataset.

TFM-05 — Diseño de la partición experimental
Aquí entramos en una etapa crítica.

Añade:

TFM-05 — Estrategia de partición agrupada del conjunto de datos
CELDA 33 — Versión de scikit-learn
```
import sklearn
from sklearn.model_selection import StratifiedGroupKFold

print("Versión de scikit-learn:", sklearn.__version__)
```

Esto nos ayuda con reproducibilidad.

CELDA 34 — Crear cinco folds agrupados y estratificados
```
# ============================================================
```

```
# TFM-05
# Validación de una partición estratificada por muestra
# ============================================================

X_dummy = np.zeros((len(labels), 1))

y = labels["is_anomaly"].to_numpy()

groups = labels["sample_num"].to_numpy()

sgkf = StratifiedGroupKFold(
n_splits=5,
shuffle=True,
random_state=SEED
)

fold_records = []
fold_indices = {}

for fold, (train_idx, test_idx) in enumerate(
sgkf.split(
X_dummy,
y,
groups
),
start=1
):

train_groups = set(
labels.iloc[train_idx]["sample_num"]
```

Esta tabla va a decirnos si los cinco folds son razonablemente comparables.

```
CELDA 35 — Comparación con la distribución global
global_anomaly_pct = (
labels["is_anomaly"].mean()
* 100
)

print(
"Porcentaje global de anomalías:",
round(global_anomaly_pct, 2),
"%"
)

fold_summary["Diferencia vs. global (pp)"] = (
fold_summary["% anómalas"]
- global_anomaly_pct
).round(2)

display(fold_summary)
```

Aquí pp significa puntos porcentuales.

Si obtenemos, por ejemplo:

Global = 35,51 %

Fold 1 = 35,0 %
Fold 2 = 36,1 %
...
sería excelente.

Pero si aparece algo como:

Fold 1 = 25 %
Fold 2 = 48 %
no aceptaremos esa partición y buscaremos otra estrategia.

CELDA 36 — Comprobar que cada muestra aparece una sola vez en test
test_group_occurrences = {}

for fold, indices in fold_indices.items():

fold_groups = (
labels.iloc[
indices["test_idx"]
]["sample_num"]
.unique()
)

for group in fold_groups:

test_group_occurrences[group] = (
test_group_occurrences.get(
group,
0
)
+ 1
)

occurrences = pd.Series(
test_group_occurrences
)

print(
"Número de muestras evaluadas:",
len(occurrences)
)

print(
"Mínimo de apariciones como test:",

El resultado correcto tiene que ser:

Número de muestras evaluadas: 150
Mínimo de apariciones como test: 1
Máximo de apariciones como test: 1

Eso demostraría que cada muestra física aparece exactamente una vez como test dentro de los cinco folds.

¿Por qué todavía no digo “80 % train / 20 % test”?
Porque quiero ver primero la tabla real.

Si los cinco folds quedan equilibrados, mi recomendación será:

80
%
→
desarrollo
80%→desarrollo
20
%
→
test final
20%→test final
utilizando el Fold 1 generado con semilla 42 como test final, no porque sea el “mejor”, sino porque es el primero definido de manera reproducible.

Eso es importante.

No escogeremos después el fold que produzca las métricas más bonitas. Hacerlo introduciría sesgo.

El 80 % restante servirá posteriormente para entrenamiento y validación mediante otra partición agrupada.

De esa manera tendremos:

150 muestras
│
├── Desarrollo ≈ 120 muestras
│ │
│ ├── entrenamiento
│ └── validación / ajuste
│
└── Test final ≈ 30 muestras
↑
NO se toca durante modelado
Y ningún sample_num podrá aparecer a ambos lados.

Una observación importante para nuestros modelos posteriores
Esta partición servirá tanto para modelos supervisados como para los de detección de anomalías.

Para modelos como One-Class SVM o determinados Autoencoders semi-supervisados podremos entrenar únicamente con las curvas normales del conjunto de desarrollo, pero evaluar después sobre normales + anómalas del test.

Para Isolation Forest, LOF y otros enfoques, decidiremos el régimen de entrenamiento de forma explícita y comparable.

No mezclaremos esas decisiones todavía.

Ejecuta ahora 33, 34, 35 y 36 y envíame principalmente la tabla de la Celda 35 y el resultado de la 36.

Si la distribución queda bien, en ese momento congelaremos el conjunto de test. A partir de ahí, no podremos utilizarlo para elegir modelos, hiperparámetros ni preprocesamiento: solo reaparecerá al final para la evaluación definitiva.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

3. Particiones agrupadas y prevención de fuga de información

3.1. Verificación del conjunto de desarrollo y test independiente

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Partición agrupada por muestra física y controles de independencia entre los grupos.

Respuesta de ChatGPT (texto conservado; selección literal): 2737/7680*100
Perfecto. TFM-05 queda validada ✅. Los resultados son suficientemente sólidos para pasar de la exploración a una partición experimental reproducible.

El Fold 1 es adecuado para nuestro test final porque contiene 30 muestras completas:

30
×
64
=
1.920
curvas
30×64=1.920 curvas
de las cuales:

1.248
normales
+
672
an
o
́
malas
=
1.920
1.248 normales+672 an
o
́
malas=1.920
y su proporción de anomalías es:

672
1920
×
100
=
35
,
00
%
1920
672

×100=35,00%
frente al 35,51 % global, una diferencia de solo −0,51 puntos porcentuales.

Además, la Celda 36 confirma lo fundamental: las 150 muestras aparecen exactamente una vez como test a lo largo de los cinco folds. Es decir, sample_num se está respetando correctamente y no tenemos fuga entre muestras.

Así queda provisionalmente nuestro experimento
Al reservar Fold 1:

Conjunto	Muestras		Curvas	Normales		Anómalas	% anómalas
Desarrollo	120	7.680	4.943	2.737	35,64 %		
Test final	30	1.920	1.248	672	35,00 %		
Total	150	9.600	6.191	3.409	35,51 %		

El porcentaje de desarrollo sale de:

2737
7680
×
100
≈
35
,
64
%
7680
2737

×100≈35,64%
Así que ambos conjuntos conservan muy bien la proporción global.

Hay un detalle que quiero auditar antes de escribir definitivamente el manifiesto: el Fold 1 contiene solo 1 de las 11 muestras completamente anómalas. Para nuestro objetivo principal binario esto no invalida el test, pero quiero comprobar que todos los tipos de anomalía están efectivamente representados en él. Es nuestra última revisión antes de “cerrar la puerta” del test.

CELDA 37 — Auditoría de categorías del test final

```
# ============================================================
# TFM-05
# Auditoría final del conjunto de test candidato
# ============================================================

FINAL_TEST_FOLD = 1

test_idx = fold_indices[FINAL_TEST_FOLD]["test_idx"]
dev_idx = fold_indices[FINAL_TEST_FOLD]["train_idx"]

test_candidate = labels.iloc[test_idx].copy()
dev_candidate = labels.iloc[dev_idx].copy()

audit_records = []

for col in category_cols:

global_presence = labels[col].fillna(0).gt(0)
test_presence = test_candidate[col].fillna(0).gt(0)
```

```
audit_records.append({
"Categoría": category_names[col],
"Global": int(global_presence.sum()),
```

Lo que quiero comprobar principalmente es:

que ninguna categoría tenga Test = 0.

No busco que todas tengan exactamente el mismo porcentaje; con categorías raras eso sería imposible.

CELDA 38 — Congelar el test final
Si la Celda 37 confirma cobertura suficiente, ejecutamos esta:

```
# ============================================================
# CONGELACIÓN DEL TEST FINAL
# ============================================================

dev_labels = labels.iloc[dev_idx].copy()
test_labels = labels.iloc[test_idx].copy()

print("DESARROLLO")
print("Muestras:", dev_labels["sample_num"].nunique())
print("Curvas:", len(dev_labels))
print(
"Normales:",
(dev_labels["is_anomaly"] == 0).sum()
)
print(
"Anómalas:",
(dev_labels["is_anomaly"] == 1).sum()
)
print(
"% anómalas:",
round(dev_labels["is_anomaly"].mean() * 100, 2)
)

print("\nTEST FINAL")
print("Muestras:", test_labels["sample_num"].nunique())
print("Curvas:", len(test_labels))
```

El resultado que esperamos es:

```
DESARROLLO
Muestras: 120
Curvas: 7680
Normales: 4943
Anómalas: 2737
% anómalas: 35.64

TEST FINAL
Muestras: 30
Curvas: 1920
Normales: 1248
Anómalas: 672
```

% anómalas: 35.0

Muestras compartidas desarrollo-test: 0
CELDA 39 — Guardar el manifiesto de partición
Esta celda es importante para reproducibilidad.

```
split_manifest = (
labels[
[
"sample_num",
"indent_num",
"path",
"is_anomaly"
]
]
.copy()
)

test_samples = set(
test_labels["sample_num"].unique()
)

split_manifest["split"] = (
split_manifest["sample_num"]
.apply(
lambda x:
"test"
if x in test_samples
else "development"
)
)

manifest_path = (
OUTPUT_DIR
/ "TFM_split_manifest_seed42.csv"
)

split_manifest.to_csv(
manifest_path,
index=False
```

Este CSV no irá como tabla en el cuerpo de la tesis. Se conserva como evidencia reproducible y eventualmente puede acompañar el notebook/material complementario.

Desde la Celda 39 cambia una regla importante
Una vez congelado:

TEST FINAL = Fold 1

no volveremos a utilizar esas 1.920 curvas para tomar decisiones.

No las usaremos para:

escoger características;

decidir normalización;
elegir algoritmos;
ajustar hiperparámetros;
seleccionar umbrales;
decidir cuál modelo “parece mejor”.
Todo eso se hará exclusivamente con las 7.680 curvas del conjunto de desarrollo.

Solo al final volveremos a abrir las 1.920 curvas de test para obtener las métricas finales.

Eso nos da una defensa metodológica bastante fuerte frente al tribunal.

Registro de evidencia
TFM-05 — VALIDADA ✅
Estrategia: partición estratificada y agrupada por muestra física.
Grupo: sample_num.
Semilla: 42.
Test final previsto: Fold 1, 30 muestras / 1.920 curvas.
Destino: Capítulo 4 → metodología experimental.
Manifiesto CSV: material reproducible, no cuerpo principal.

Ejecuta primero 37. Si todas las categorías aparecen en el test, ejecuta 38 y 39 y mándame esas salidas.

Después empieza TFM-06, que es un cambio importante: construiremos por primera vez la matriz numérica de las curvas y decidiremos qué información verá realmente cada algoritmo.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

3.2. Auditoría de la celda 37 y separación de los folds

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Comprobación de particiones y asignación de muestras físicas según los resultados mostrados.

Respuesta de ChatGPT (texto conservado; selección literal): Este es el resultado de la celda 37.
Sí, la Celda 37 pasa la auditoría y podemos congelar el Fold 1 como test final.

Lo más importante es que las ocho categorías aparecen en test; no tenemos ninguna con Test = 0. Además, las categorías principales conservan una representación razonablemente cercana a la global:

force_offset: 296 curvas; 15,42 % en test vs. 14,04 % global.
unusual_unloading_curvature: 205; 10,68 % vs. 10,30 %.
unusual_loading_curvature: 69; 3,59 % vs. 2,36 %.
tip_displacement_decreases_at_holding: 42; 2,19 % vs. 2,32 %.
displacement_offset: 17; 0,89 % vs. 1,45 %.
too_deep: 13; 0,68 % vs. 0,88 %.
other: 9; 0,47 % vs. 0,32 %.
La desviación mayor está en “muestra completamente anómala”: 64 curvas, equivalentes a una única muestra completamente anómala, con 3,33 % del test frente a 7,33 % global. Pero esto no invalida nuestro test binario, porque el objetivo principal es normal vs. anómala y el conjunto completo mantiene un 35,00 % de anomalías frente al 35,51 % global.

Y aquí hay una decisión metodológica importante: no debemos buscar otro fold solamente porque reproduzca mejor cada subtipo. Ya fijamos el Fold 1 antes de mirar estas distribuciones específicas y todas las categorías están representadas. Elegir ahora otro porque “queda más bonito” introduciría sesgo.

Eso sí, cuando analicemos resultados por tipo específico de anomalía, tendremos que ser cautos con categorías raras. Por ejemplo, 13 casos de too_deep o 9 de other no permiten sacar conclusiones tan robustas como 296 casos de force_offset. Pero para la evaluación binaria global no es un problema.

Por tanto: ejecuta Celda 38 y 39
En la Celda 38 debemos obtener aproximadamente:

Perfecto, estás en el lugar correcto.

En el área grande donde dice “Enter file contents here”, pega exactamente esto:

Dataset original: descargar desde la fuente científica
anomaly_detection_on_nanoindentation*.zip
nanoindentation_dataset/

Resultados generados por el notebook
resultados/
*.joblib

Entornos / cachés
.ipynb_checkpoints/
__pycache__/
.DS_Store
Después:

Confirma que arriba el nombre sea exactamente:

.gitignore
Pulsa Commit changes arriba a la derecha.

GitHub te mostrará una ventana para el mensaje del commit. Puedes poner:

Add gitignore for dataset and generated files
Confirma nuevamente con Commit changes.

Con eso, el repositorio queda protegido para que no subamos accidentalmente el ZIP del dataset ni archivos temporales.

Después regresa a la página principal del repositorio y mándame una captura. Ahí verificamos que aparezcan estos cinco elementos:

manifests/
.gitignore
README.md
requirements.txt
TFM_Nanoindentacion_Deteccion_Anomalias.ipynb
Si están los cinco, el siguiente paso será comprobar que el README se vea correctamente y probar que el notebook se pueda abrir desde GitHub.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

4. Publicación y preservación del código

4.1. Confirmación de commit en GitHub

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Orientación operativa sobre GitHub para registrar archivos técnicos.

Respuesta de ChatGPT (texto conservado; selección literal): Sí, está perfecto.

Deja seleccionada la opción:

Commit directly to the main branch

No necesitamos crear una rama nueva ni un pull request para este caso.

El mensaje de commit puede quedarse tal como está:

Create .gitignore

y la descripción:

Add gitignore for dataset and generated files

Ahora pulsa el botón verde Commit changes.

Cuando termine, vuelve a la vista principal del repositorio y comprueba que aparezca .gitignore junto a los demás archivos. Si me envías la siguiente captura, revisamos el repositorio completo antes de añadir la URL al TFM.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

4.2. Verificación del archivo .gitignore

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Revisión de la organización y exclusiones del repositorio.

Respuesta de ChatGPT (texto conservado; selección literal): listo
Perfecto. Con eso ya debería estar creado también el .gitignore.

Ahora hagamos la última comprobación manual del repositorio antes de meter la URL en la tesis:

Vuelve a la pestaña Code del repositorio.
Debes ver estos cinco elementos:
manifests/
.gitignore
README.md
requirements.txt
TFM_Nanoindentacion_Deteccion_Anomalias.ipynb
Abre README.md y confirma que GitHub lo muestra correctamente debajo de los archivos.
Abre el .ipynb y confirma que GitHub permite visualizar el notebook.
Copia esta dirección del repositorio:

https://github.com/hectoarea91/TFM-anomaly-detection-material-curves
Muy importante: abre una ventana Incógnito/InPrivate, pega esa URL y verifica que puedas entrar sin iniciar sesión. Eso demuestra que realmente es público.
Por lo que vi en tus capturas, el repositorio fue creado como Public, pero no puedo confirmar desde aquí que la carga final sea accesible externamente porque la página pública todavía no aparece indexada en la búsqueda web. Por eso la prueba en incógnito es la comprobación definitiva.

Si abre correctamente, ya podemos incorporar en 4.3.1 Entorno de trabajo y reproducibilidad este párrafo:

Y en el Anexo A, reemplazamos:

[INCORPORAR AQUÍ LA URL DEFINITIVA DE GITHUB]

por:

Repositorio público del código: https://github.com/hectoarea91/TFM-anomaly-detection-material-curves

La Guía exige que, cuando se incluya código, el repositorio sea de acceso público y se referencie mediante su URL, así que con esto estaríamos cumpliendo ese requisito.

Haz primero la prueba en incógnito. Si abre bien, me dices “abre correctamente” y pasamos directamente a integrar definitivamente los Anexos A y B en la memoria.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

4.3. Revisión de acceso público y de los archivos del repositorio

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Asesoramiento para asegurar la consulta y reproducibilidad del notebook.

Respuesta de ChatGPT (texto conservado; selección literal): LISTO
Perfecto. Tomo entonces como verificado que el repositorio abre correctamente en modo incógnito y que, por tanto, es público y accesible. Ya podemos incorporarlo formalmente al TFM.

Primero, en 4.3.1. Entorno de trabajo y reproducibilidad, añade al final este párrafo:

Con el propósito de favorecer la reproducibilidad del desarrollo experimental, el notebook utilizado para la ejecución del procedimiento, junto con los archivos de configuración y los manifiestos asociados a las particiones de los datos, se encuentra disponible en un repositorio público de GitHub: https://github.com/hectoarea91/TFM-anomaly-detection-material-curves. El repositorio incluye las dependencias necesarias para reproducir el entorno utilizado y mantiene explícitamente scikit-learn 1.6.1, debido a la influencia de esta versión sobre la generación de las particiones mediante StratifiedGroupKFold.

Esto cubre directamente el requisito de la Guía de que el código incluido en el trabajo se encuentre en un repositorio público y se referencie mediante URL.

Después de Referencias bibliográficas, crea una página nueva y añade los anexos.

Anexo A. Reproducibilidad del desarrollo experimental
El desarrollo experimental del presente Trabajo Fin de Máster se realizó mediante un notebook de Python ejecutado principalmente en Google Colab. Con el propósito de favorecer la trazabilidad y reproducibilidad del análisis, el código utilizado se encuentra organizado en un repositorio público que contiene el notebook principal, los archivos de

configuración y los manifiestos empleados para documentar las particiones del conjunto de datos.

El conjunto experimental utilizado corresponde al dataset público de curvas carga-desplazamiento de nanoindentación desarrollado por Henriksen et al. (2025), disponible mediante el DOI 10.18126/yvgn-w386. Los archivos originales del conjunto de datos no se incorporan al repositorio del TFM y deben descargarse directamente desde su fuente científica.

La ejecución original se realizó utilizando una semilla igual a 42 y scikit-learn 1.6.1. Esta versión se mantiene explícitamente fijada en el notebook debido a que el comportamiento de StratifiedGroupKFold con shuffle=True puede variar entre versiones de la biblioteca. Por esta razón, la versión del entorno constituye un elemento necesario para reproducir exactamente las particiones y métricas documentadas en la memoria.

El procedimiento experimental comprendió la carga y validación del conjunto de datos, el análisis exploratorio, la construcción de la variable binaria normal/anómala, la reserva de un conjunto de prueba independiente, la construcción de diferentes representaciones de las curvas, la validación cruzada agrupada, la comparación y ajuste de los modelos, la selección del umbral y la evaluación final.

La variable sample_num se utilizó como identificador de grupo durante las particiones. De esta manera, las curvas correspondientes a una misma muestra física permanecieron siempre dentro de una única partición y no se compartieron muestras entre desarrollo y prueba.

El conjunto de prueba independiente quedó compuesto por 30 muestras físicas y 1.920 curvas, mientras que el conjunto de desarrollo quedó formado por 120 muestras y 7.680 curvas. La asignación definitiva se conserva mediante el archivo TFM_split_manifest_seed42.csv.

Adicionalmente, el notebook genera el archivo TFM_internal_cv_manifest_seed42.csv, destinado a documentar el fold de validación asignado a cada observación del conjunto de desarrollo. Esta información permite conservar explícitamente las cinco particiones empleadas durante la validación cruzada interna.

El repositorio contiene también un archivo requirements.txt con las dependencias principales, un archivo .gitignore destinado a evitar la incorporación accidental del dataset y de archivos derivados, y un archivo README.md con las instrucciones necesarias para ejecutar el notebook y localizar el conjunto de datos.

Repositorio público del código:
https://github.com/hectoarea91/TFM-anomaly-detection-material-curves

La disponibilidad conjunta del notebook, la versión del entorno, la semilla y los manifiestos de partición proporciona los elementos necesarios para reproducir el procedimiento experimental y verificar la correspondencia entre el código ejecutado y los resultados presentados en la memoria.

Después, en página nueva:

Anexo B. Resultados complementarios de optimización y análisis
Este anexo reúne información complementaria generada durante las etapas de ajuste y análisis de los modelos. Su finalidad es aportar trazabilidad adicional al desarrollo experimental sin incorporar en el cuerpo principal de la memoria todas las salidas intermedias del notebook.

B.1. Sensibilidad de Local Outlier Factor al número de vecinos
Durante el ajuste de Local Outlier Factor se evaluaron los valores:

k = {5, 10, 15, 20, 30, 40, 60, 80, 100}

Los resultados mostraron una región de comportamiento estable aproximadamente entre 15 y 30 vecinos. Entre las configuraciones situadas en esta región se obtuvieron los siguientes valores:

n_neighbors	ROC-AUC medio	Average Precision medio
15	0,9487	0,9340
20	0,9492	0,9338
30	0,9495	0,9326

La configuración final se estableció en n_neighbors = 20. Para hacer reproducible esta decisión, el notebook considera como candidatas las configuraciones cuya Average Precision se encuentra a una distancia máxima de 0,001 respecto al mejor resultado y, entre ellas, selecciona la de mayor ROC-AUC.

B.2. Ajuste de One-Class SVM

Para One-Class SVM se evaluaron los valores:

$\nu = \{0{,}01;\ 0{,}025;\ 0{,}05;\ 0{,}10;\ 0{,}15\}$

y los multiplicadores:

$\gamma_{mult} = \{0{,}25;\ 0{,}50;\ 1;\ 2;\ 4\}$

La configuración seleccionada fue:

$\nu = 0{,}10$

$\gamma = 4 \times \gamma_{base}$

con:

$\gamma_{base} = 0{,}052632$

$\gamma_{final} = 0{,}210526$

Esta configuración alcanzó durante la validación interna un ROC-AUC medio de 0,9466 y una Average Precision media de 0,9296.

B.3. Selección del umbral

Los umbrales binarios se determinaron utilizando exclusivamente las puntuaciones obtenidas sobre curvas normales del conjunto de desarrollo. Se evaluaron los percentiles:

$q = \{0{,}90;\ 0{,}925;\ 0{,}95;\ 0{,}975;\ 0{,}99\}$

El criterio principal de selección fue F1, complementado con Balanced Accuracy, precisión, recall y especificidad. Para ambos modelos se seleccionó:

$q = 0{,}95$

Los umbrales definitivos obtenidos después del entrenamiento final fueron:

$LOF = 1{,}309295$

$\text{One-Class SVM} = 0{,}801311$

Estos valores quedaron fijados antes de realizar la evaluación sobre el conjunto de prueba independiente.

B.4. Evaluación final

Los resultados obtenidos sobre las 1.920 curvas del conjunto de prueba independiente fueron:

Métrica	LOF	One-Class SVM	
ROC-AUC		0,9352	0,9375
Average Precision		0,9098	0,9123
Precisión		0,8323	0,8147
Recall	0,7827	0,8438	
Especificidad		0,9151	0,8966
F1	0,8067	0,8289	
Balanced Accuracy		0,8489	0,8702

Las matrices de confusión fueron:

LOF: TN = 1.142; FP = 106; FN = 146; TP = 526.

One-Class SVM: TN = 1.119; FP = 129; FN = 105; TP = 567.

One-Class SVM presentó el mejor equilibrio global entre sensibilidad y clasificación correcta de curvas normales, mientras que LOF mostró mayor precisión y especificidad. Estas diferencias corresponden al dominio experimental estudiado y no deben interpretarse como evidencia de superioridad universal de un modelo frente al otro.

B.5. Casos representativos
Como parte del análisis de errores se inspeccionaron ejemplos correspondientes a las cuatro situaciones de una clasificación binaria utilizando One-Class SVM:

verdadero positivo: muestra 6404, indentación 3, puntuación aproximada 7,7925;

verdadero negativo: muestra 6783, indentación 37, puntuación aproximada −3,6025;

falso positivo: muestra 6815, indentación 11, puntuación aproximada 0,8046;

falso negativo: muestra 6270, indentación 40, puntuación aproximada 0,7436.

El falso positivo se situó prácticamente sobre el umbral de 0,801311, mientras que el falso negativo correspondió a una curva que contenía la etiqueta de curvatura inusual durante la descarga. Debido a la estructura multietiqueta del conjunto de datos, esta asociación no debe interpretarse como una relación causal.

Hay además una pequeña corrección de coherencia: en 1.4 Estructura del documento actualmente utilizas la palabra “apéndices”. Como la Guía Docente utiliza expresamente “Anexos”, recomiendo reemplazar allí “apéndices” por “anexos”. La Guía permite incorporarlos cuando sean necesarios y los excluye del intervalo de 40–80 páginas, aunque sí cuentan dentro del máximo global de 30.000 palabras.

Finalmente, después de añadir los anexos recuerda actualizar la tabla de contenido completa. La estructura final debería terminar con:

5. Conclusiones y desarrollos futuros
Referencias bibliográficas
Anexo A. Reproducibilidad del desarrollo experimental
Anexo B. Resultados complementarios de optimización y análisis

Con esto tendremos cerrados código + repositorio + anexos + reproducibilidad. Después de incorporarlos al Word, lo recomendable es que me cargues esa nueva versión y hacemos una última auditoría breve antes de enviársela al profesor asesor.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

5. Revisión académica del documento

5.1. Auditoría de los anexos y formato matemático

Tipo de registro: Extracto técnico de respuesta. La pregunta que lo originó no se conserva de forma inequívoca en este archivo; no se reconstruye ni se atribuye una pregunta inventada.

Objeto de la consulta: Revisión de anexos, formatos de ecuaciones y consistencia de notación.

Respuesta de ChatGPT (texto conservado; selección literal): Sí, ya revisé visualmente las cinco páginas de anexos. La estructura está bien y los anexos aportan valor real: no son relleno, sino que documentan reproducibilidad, tuning, umbral y resultados complementarios. Antes de darlos por cerrados, haría solo algunos ajustes de presentación y consistencia.

El más importante está en el título del Anexo A. En la captura aparece:

ANEXO A – Reproducción en el desarrollo experimental

Debe quedar:

ANEXO A – Reproducibilidad del desarrollo experimental

Ese es el término correcto y además coincide con la finalidad del anexo y con el lenguaje que venimos utilizando en 4.3.1.

En el Anexo B, revisaría la notación matemática de B.2. En la captura algunas variables parecen haberse convertido visualmente en una v o quedar poco claras. Yo mantendría siempre:

v = {0,01; 0,025; 0,05; 0,10; 0,15}
γmult = {0,25; 0,50; 1; 2; 4}
v = 0,10
γ = 4 × γbase
γbase = 0,052632
γfinal = 0,210526

Si Word te da problemas con las letras griegas, es preferible escribir nu, gamma_base y gamma_final en fuente monoespaciada que dejar símbolos ambiguos.

También corregiría la Tabla de B.4, porque queda partida entre las páginas 90 y 91. En la página 91 aparecen únicamente Especificidad, F1 y Balanced Accuracy, sin repetir el encabezado de la tabla. No está mal técnicamente, pero visualmente parece una tabla incompleta. La mejor opción es intentar que toda la tabla pase a la página 91. Si no cabe, activa en Word:

Propiedades de tabla → Fila → Repetir como fila de encabezado en cada página

y desactiva:

Permitir dividir filas entre páginas.

En el Anexo A, la división entre las páginas 87 y 88 sí me parece correcta. La primera página termina después de explicar sample_num y la segunda continúa con la composición desarrollo/test, manifiestos y repositorio. No se siente como un corte arbitrario.

Hay además un detalle de estilo: tienes bastantes palabras marcadas con subrayado rojo de Word —`dataset`, Google Colab, StratifiedGroupKFold, One-Class SVM, Average Precision, etc.—. No significa que estén mal; el corrector simplemente no reconoce esos términos. No tienes que cambiar la terminología científica por eso. Para la versión PDF final esos subrayados no aparecerán.

La URL del repositorio está muy bien ubicada en el Anexo A y se distingue claramente:

https://github.com/hectoarea91/TFM-anomaly-detection-material-curves.git

Sin embargo, yo eliminaría el .git final en la memoria y usaría la dirección más amigable para lectura humana:

https://github.com/hectoarea91/TFM-anomaly-detection-material-curves

El enlace sin .git lleva a la página del repositorio; el que termina en .git se utiliza principalmente para operaciones Git como clonar.

También veo bien que el Anexo B termine con los cuatro casos representativos. Los valores que aparecen en tu captura coinciden con los que verificamos en el notebook:

TP: muestra 6404, indentación 3, ≈ 7,7925.
TN: muestra 6783, indentación 37, ≈ −3,6025.
FP: muestra 6815, indentación 11, ≈ 0,8046.
FN: muestra 6270, indentación 40, ≈ 0,7436.
Y el cierre sobre la naturaleza multietiqueta evita correctamente hacer una afirmación causal que los datos no permiten.

Mi evaluación ahora mismo sería:

Anexo A: contenido aprobado; corregir título y URL.
Anexo B: contenido aprobado; mejorar notación v/γ y evitar que la Tabla B.4 quede partida.

Después de esos ajustes, yo no añadiría más anexos. Ya tienes suficiente evidencia complementaria sin inflar innecesariamente la tesis.

Cuando hagas esas tres correcciones, el paso que recomiendo es que me vuelvas a cargar el Word completo ya con los anexos. Ahí hacemos una última auditoría de portada → índices → Resumen/Abstract → cuerpo → referencias → anexos → URL del repositorio y, si todo queda consistente, ya puedes enviárselo al profesor asesor como borrador prácticamente final.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.

6. Consulta verificada sobre GitHub y Google Colab

6.1. Consulta sobre publicación y ejecución del notebook

Pregunta literal del estudiante: Listo, antes de enviarte el documento para revisión del miemo, ahora se puede que el codigo se pueda ejecutar en github o se podria pasar o anexar el notebook de google para que se vea el prodeicmiento y los resultados de ejecución del desarrollo del proyecto?

Objeto de la consulta: Consulta sobre la disponibilidad y ejecución del notebook y la presentación de sus resultados mediante un repositorio.

Respuesta de ChatGPT (texto conservado; selección literal): Sí, y para tu TFM la mejor solución es usar las dos cosas:

GitHub como repositorio público y evidencia reproducible.
Google Colab como entorno donde el evaluador puede ejecutar el notebook.
GitHub renderiza los archivos .ipynb como una vista estática: permite ver celdas de texto, código y las salidas que estén guardadas, pero no ejecuta interactivamente las celdas como Colab.
Google Colab, en cambio, puede abrir directamente un notebook almacenado en GitHub y ejecutarlo en la nube.

Lo que recomiendo para nuestro repositorio
Mantendría dos notebooks:

TFM_Nanoindentacion_Deteccion_Anomalias.ipynb
TFM_Nanoindentacion_Deteccion_Anomalias_EJECUTADO.ipynb
El primero sería el que ya subimos: limpio, reproducible y sin salidas anteriores. Es el adecuado para que otra persona lo ejecute desde cero.

El segundo sería una captura documental de la ejecución completa, conservando las salidas: tablas, métricas, matrices de confusión, figuras y resultados. Así, un profesor que simplemente entre a GitHub puede recorrer todo el procedimiento y ver qué produjo cada celda sin necesidad de ejecutar nada. Colab permite compartir el notebook conservando texto, código y outputs, salvo que se haya activado explícitamente la opción de omitir salidas.

Esto me parece especialmente bueno para tu defensa porque permite demostrar simultáneamente:

“Aquí está el código reproducible.”
“Aquí está la ejecución que generó los resultados presentados en la memoria.”

Y añadimos un botón «Open in Colab»
Podemos poner al principio del README.md un botón que lleve directamente desde GitHub a Colab:

[![Open In Colab](https://colab.research.google.com/assets/colab-badge.svg)](https://colab.research.google.com/github/hectoarea91/TFM-anomaly-detection-material-curves/blob/main/TFM_Nanoindentacion_Deteccion_Anomalias.ipynb)
Al pulsarlo, abrirá tu notebook en Google Colab y el usuario podrá ejecutar las celdas. Colab admite precisamente cargar notebooks directamente desde GitHub.

¿Se puede ejecutar directamente «dentro de GitHub»?
No de la misma manera que en Colab. GitHub puede mostrar el notebook pero no ofrece un kernel interactivo para pulsar Play en cada celda.

Técnicamente podríamos configurar GitHub Actions para ejecutar automáticamente el notebook, pero para este TFM no lo recomiendo. Añade complejidad innecesaria, especialmente porque el dataset está alojado externamente y hemos fijado un entorno concreto con scikit-learn==1.6.1.

La combinación GitHub + Colab es bastante más clara.

Cómo generamos ahora el notebook ejecutado
Lo haría desde el notebook que acabamos de preparar para GitHub:

En GitHub abre:
TFM_Nanoindentacion_Deteccion_Anomalias.ipynb
Ábrelo en Colab.
Comprueba que está usando scikit-learn 1.6.1.
Proporciona el ZIP del dataset.
Ejecuta Runtime → Run all.
Espera a que termine hasta los resultados finales.
Comprueba que aparecen nuevamente:
9.600 curvas;
test de 1.920;
LOF ROC-AUC 0,9352;
OCSVM ROC-AUC 0,9375;
TP OCSVM = 567;
umbral OCSVM = 0,801311.
Guarda una copia denominada:
TFM_Nanoindentacion_Deteccion_Anomalias_EJECUTADO.ipynb
Súbela al mismo repositorio.
No reemplazaría el notebook limpio. Los conservaría ambos.

La estructura final quedaría especialmente sólida:

```
TFM-anomaly-detection-material-curves/
│
├── README.md
├── .gitignore
├── requirements.txt
│
├── TFM_Nanoindentacion_Deteccion_Anomalias.ipynb
├── TFM_Nanoindentacion_Deteccion_Anomalias_EJECUTADO.ipynb
│
└── manifests/
└── TFM_split_manifest_seed42.csv
```

Y actualizaríamos el README indicando claramente:

Notebook reproducible: versión limpia destinada a ejecutar el procedimiento desde el inicio.
Notebook ejecutado: versión que conserva las salidas correspondientes a la ejecución utilizada para verificar los resultados documentados en el TFM.

Esto es mejor que meter capturas del código en los anexos: los anexos explican qué se hizo y cómo reproducirlo, mientras GitHub conserva el código completo y la evidencia de ejecución.

Si quieres, hagamos ahora mismo el notebook ejecutado: te guío desde tu repositorio para abrirlo en Google Colab, ejecutarlo con el dataset y volver a guardarlo en GitHub sin sobrescribir el notebook limpio.

Perfecto. Lo hacemos uno por uno, sin saltarnos pasos.

Paso 1 — Abrir el notebook limpio desde GitHub
Entra a tu repositorio:

https://github.com/hectoarea91/TFM-anomaly-detection-material-curves

Luego:

Haz clic en el archivo TFM_Nanoindentacion_Deteccion_Anomalias.ipynb.
Espera a que GitHub termine de mostrar el notebook.
No edites nada todavía.
Cuando estés viendo el notebook abierto dentro de GitHub, envíame una captura de esa pantalla y seguimos con el Paso 2: abrirlo correctamente en Google Colab.

Nota editorial: transcripción seleccionada. Puede incluir recomendaciones o resultados que deben contrastarse con el notebook y la memoria originales. No constituye certificación independiente de la ejecución.